divisions become apparent only as the data is processed. This is particularly apparent with recursive algorithms such as Quicksort; they therefore need a different approach.

8.1.2. Dividing data recursively

The Quicksort algorithm has two basic steps: partition the data into items that come before or after one of the elements (the pivot) in the final sort order and recursively sort those two "halves." You can't parallelize this by simply dividing the data up front, because it's only by processing the items that you know which "half" they go in. If you're going to parallelize such an algorithm, you need to make use of the recursive nature. With each level of recursion there are *more* calls to the `quick_sort` function, because you have to sort both the elements that belong before the pivot *and* those that belong after it. These recursive calls are entirely independent, because they access separate sets of elements, and so are prime candidates for concurrent execution. Figure 8.2 shows such recursive division.

In chapter 4, you saw such an implementation. Rather than just performing two recursive calls for the higher and lower chunks, you used `std::async()` to spawn asynchronous tasks for the lower chunk at each stage. By using `std::async()`, you ask the C++ Thread Library to decide when to actually run the task on a new thread and when to run it synchronously.

Figure 8.2. Recursively dividing data

